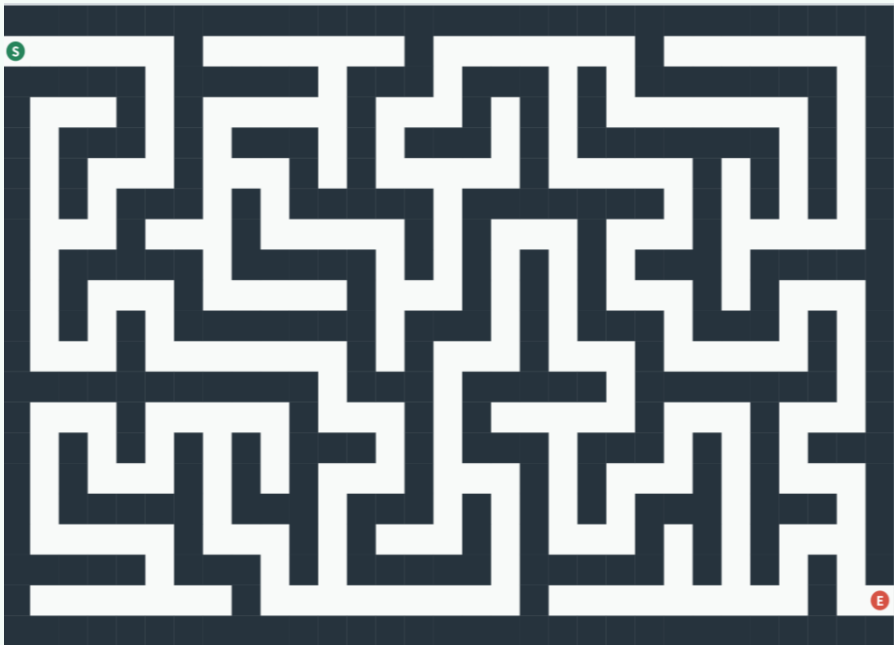




无需建造一座迷宫，因为整个宇宙本身就是一座迷宫。

How will I ever get out of this
labyrinth!

DSA课程设计

走迷宫

组员：冯卓源、李斯羽

汇报人：李斯羽

DSA @ Spring 2026

设计梗概

❖ 【实现】迷宫生成、路径搜索、图形化演示

- 随机矩形迷宫
- 蒙德里安名画迷宫
- 圆形迷宫 ...

❖ 【扩展内容】基于元胞自动机（CA）和进化策略自动生成迷宫

- 在多族 CA 规则空间中搜索
- 候选规则先演化出二值网格
- 建立打分规则判断该网格有多像迷宫
- 把分数作为进化策略的适应度，保留高分规则、变异并产生下一代

Hogwarts

Slytherin

问题分析

❖ 需要解决以下问题

- 使用合适的数据结构表示墙、通路、入口和出口
- 自动生成一个连通的随机迷宫
- 找到**步数最少**的路径
- 通过图形界面动态展示搜索顺序和最终路径
- 将蒙德里安几何画作拆分成多个房间，保证路径至少经过三个房间
- 生成圆形等非矩形迷宫，并支持出口位于迷宫内部

Hogwarts

Slytherin

问题分析

❖ 问题建模

- 迷宫可以抽象为图 $G = (V, E)$
- 每个可通行位置对应一个顶点，相邻且连通的位置之间存在一条边
- 寻路问题就是在图中寻找一条从起点 S 到终点 E 的路径。
- 矩形迷宫中的顶点由 (row, col) 表示，相邻关系是上、右、下、左四个方向
- 圆形迷宫，将顶点改为 $(ring, sector)$ ，分别表示环号和扇区号
- 同环相邻扇区以及内外环对应扇区之间构成边

hogwarts

Slytherin

数据结构设计与实现

❖ C++ 控制台版本使用 `vector<string>` 保存迷宫

❖ 网页版本使用二维数组保存每个格子的状态

- `Point { r, c }`: 表示矩形迷宫中的坐标值网格
- `grid`: 二维迷宫矩阵
- `visited / seen`: 记录顶点是否已经访问, 避免重复搜索
- DFS 与 BFS...

Hogwarts

```
class Maze {
public:
    Maze(vector<string> grid, Point start, Point exit)
        : grid_(move(grid)), start_(start), exit_(exit) {}

    static Maze demo() {
        vector<string> grid = {
            "#####",
            "S   #   #",
            "### # ### #",
            "#   #   #",
            "# ### # ###",
            "#       E",
            "#####",
        };
        return Maze(grid, {1, 0}, {5, 10});
    }

    static Maze randomMaze(int rows, int cols) {
        rows = normalizeSize(rows);
        cols = normalizeSize(cols);

        vector<string> grid(rows, string(cols, '#'));
        Point start{1, 0};
        Point innerStart{1, 1};
        Point exit{rows - 2, cols - 1};
        Point innerExit{rows - 2, cols - 2};
    }
};
```

复杂度分析

❖ 时间复杂度与空间复杂度分析

- 记迷宫行数为 R ，列数为 C ，可通行顶点总数为 V ，相邻可通行顶点之间的连通边数记为 E
- 考虑到顶点至多连接四条边，因此二者同阶

❖ 迷宫生成

- DFS生成迷宫时，每个候选单元最多被访问一次，时空复杂度均为 $O(RC)$

❖ 寻路与回溯

- BFS的时空复杂度仅与路径长度 L 有关，均为 $O(L)$

Hogwarts

Slytherin

结果分析

❖ 算法结果与比较

- 在同一迷宫上, BFS 得到的路径长度不大于 DFS
- 蒙德里安迷宫验证了普通图搜索可以应用的实际情况
- 圆形迷宫验证了邻接关系改变后, 同一套搜索思想可以用于更广义的非矩形结构

❖ 存在的不足

- $O(RC)$

Hogwarts

Slytherin

成员分工

这个项目的选择一开始是冯卓源同学提出的，他找到了关于元胞自动机的视频，受到启发希望能在在此基础上实现迷宫生成与打分的自动化。我负责的是朴素的DFS实现方法与前端设计，他则负责一开始提出的元胞自动机的实现。受限于能力、硬件的不足，最终成果可能还存在许多不完善的地方，希望老师多多包涵。

Hogwarts

Slytherin

扩展：CA + ES 自动生成迷宫

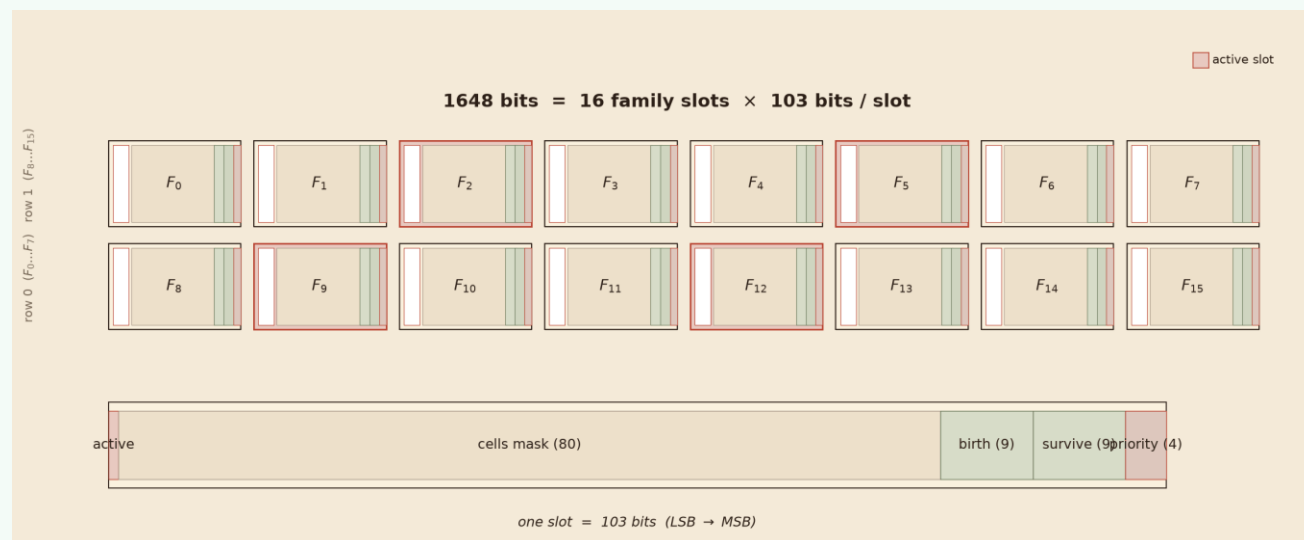
- ❖ Conway B/S 字符串只有 2^{18} 种, 几乎搜不到细长连通迷宫
- ❖ 改用多族 CA (FamilyMask), 染色体 = 2^{1648} bit
- ❖ 用进化策略搜索:
 - 候选规则演化出 40×60 二值网格
 - 用 maze_quality 打分, 作为 fitness
 - 保留高分规则, 变异产生下一代
- ❖ WebGPU 加速: 单条 6.9 min, 160 run 共 21.92 h

Hogwarts

Slytherin

FamilyMask 多族 CA 染色体

- ❖ B/S 字符串推广到最多 16 族
- ❖ 每族 103 bit:
 - 80-bit cells mask
 - 9-bit birth
 - 9-bit survive
 - 4-bit priority
- ❖ 染色体总长 2^{1648} bit
- ❖ 演化时按优先级遍历, 第一个匹配族决定下一格
- ❖ 单族时退化为经典 B/S



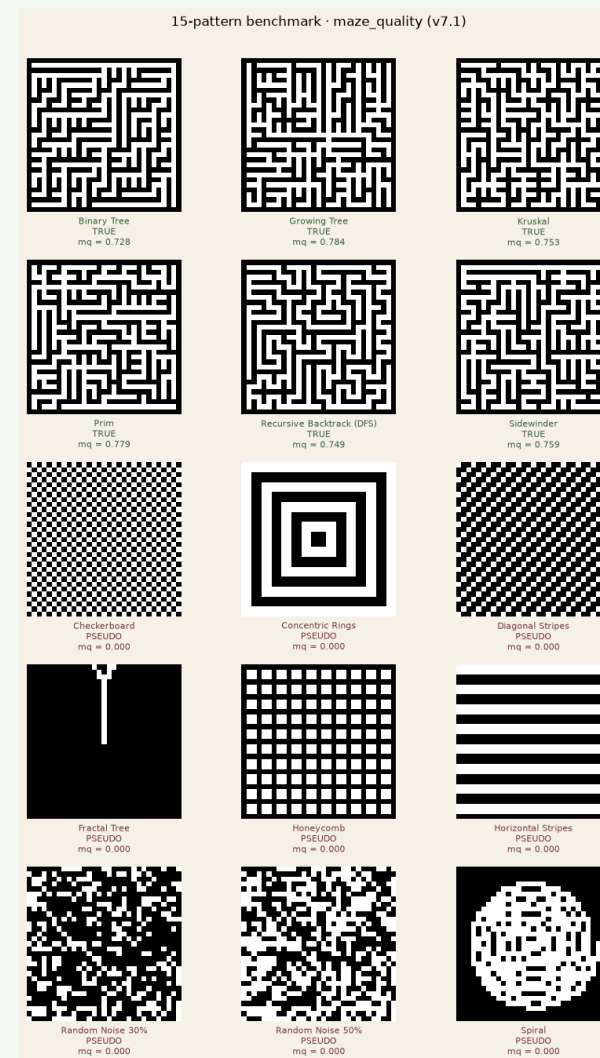
Hogwarts

Slytherin

maze_quality 十维迷宫度量

- ❖ 10 个子度量, 值域 [0,1]
 - 5 拓扑: corridors / diffusion / junctions / connectivity / outer ring
 - 5 多样性: local blocks / self-symmetry / adjacency pairs / block uniqueness / long-run ratio
- ❖ 加权聚合 → min 平衡 → 三角墙比门控 [0.4, 0.6]
- ❖ **15-pattern 测试: 0/15 误判**
 - TRUE mean 0.759, PSEUDO mean 0
- ❖ 扩展到 239 grids (多算法 × 多种子):
 - **67.6% 全对, 94.6% 对 14/15 以上**
 - 唯一失败模式: Spiral 高转数种子

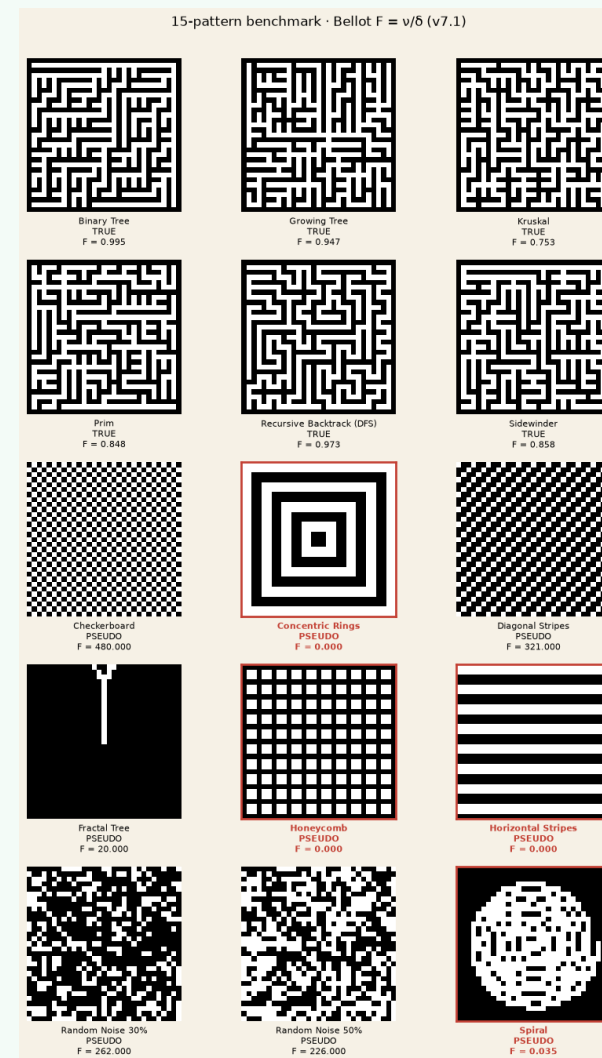
Hogwarts



Bellot 2021 F 的 cell-based 复刻

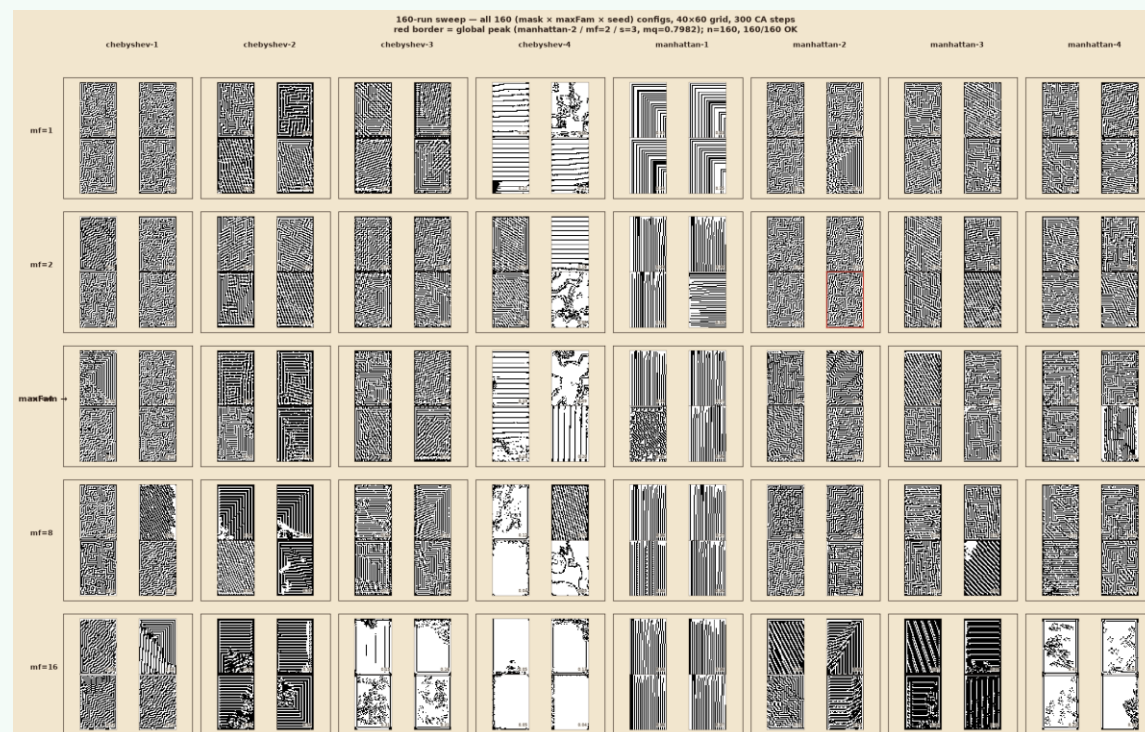
- ❖ Bellot 2021 $F = v / \delta$
- ❖ 本文 cell-based 复刻, 数值与原文 wall-based 方向相反
- ❖ 15-pattern 结果:
 - TRUE mean $F = 0.896$
 - PSEUDO mean $F = 145.448$
- ❖ 按 Bellot 假说判 4/9 误判
 - 失败模式: Spiral / HStripes / CRings / Honeycomb
 - 都是 $v=0$ 类, Bellot 假说本身不适用
- ❖ 方向结论一致: F 小 = 更像 maze

Hogwarts



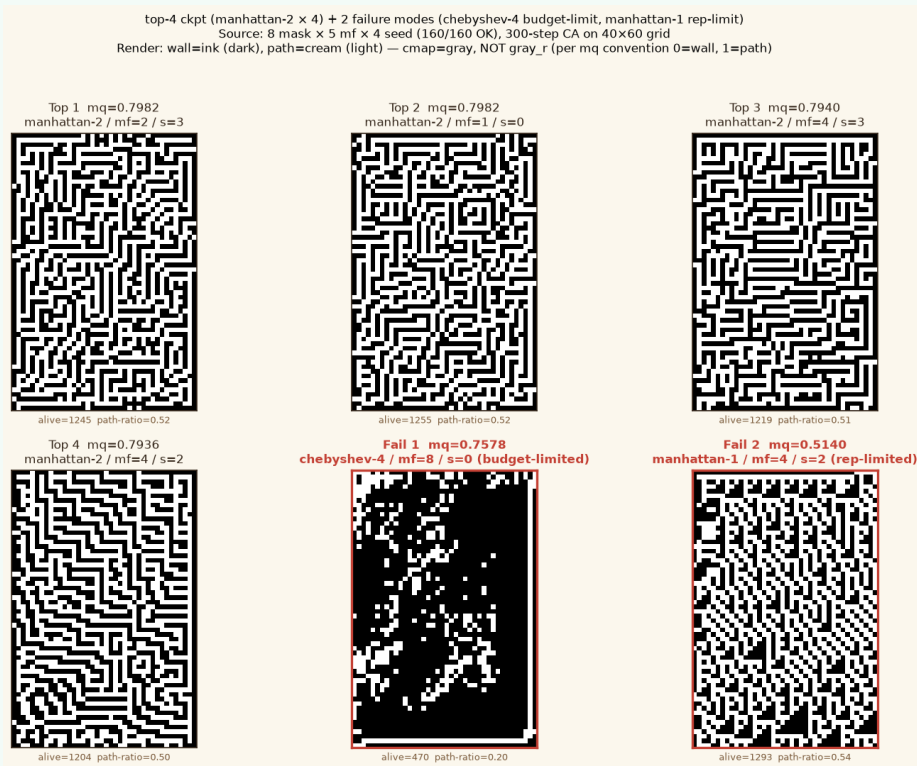
ES 搜索: 160 run 全交叉扫描

- ❖ $8 \text{ mask} \times 5 \text{ maxFam} \times 4 \text{ seed} = 160 \text{ run}$, 总耗时 21.92 h
- ❖ Top-1 score 0.7982 (manhattan-2 / maxFam=2 / seed=3)
- ❖ Top-10 全 manhattan ($8 \times \text{mf}=2 + 2 \times \text{mf}=4$)
- ❖ 跨系列 p-value = 0.0007



Slytherin

Top-1 与 maze_seal 流水线



- ❖ 左: Top-4 最佳 + 2 失败对照
- ❖ 右: 最佳 ckpt 三视图 (原图 $\rightarrow$ seal 后 $\rightarrow$ 带路径)
- ❖ maze_seal: 12 个 ckpt 打包, clone 后可直接复现